Solution: 132 Pattern

Problem Statement

Given an array `nums`, containing `N` integers.

A `132 pattern` consists of three numbers, say x, y, and z, where `x < z` and `z < y`. This is often referred to as a '132' pattern because if we represent x, y, and z as 1, 3, and 2, respectively, it mimics the positional pattern in '132'.

Return `true` if such a pattern exists within a given sequence of numbers. Otherwise, return `false`.

Examples

1. **Example 1**:
    - Input: nums = [3, 5, 0, 3, 4]
    - Expected Output: True
    - Justification: Here, 3 < 4 and 4 < 5, forming a '132' pattern with the numbers 3, 5, and 4.
2. **Example 2**:
    - Input: nums = [1, 2, 3, 4]
    - Expected Output: False
    - Justification: The sequence is in ascending order, and no '132' pattern is present.
3. **Example 3**:
    - Input: nums = [9, 11, 8, 9, 10, 7, 9]
    - Expected Output: True
    - Justification: The pattern is formed with 9 < 10 and 10 < 11.

Solution

The approach for identifying the "132" pattern is centered on efficiently scanning the array to detect the specific sequence. This involves maintaining a variable for the potential third element of the pattern, which is initially set very low. The key is to iterate through the array in reverse. During this traversal, if an element is found that is smaller than this potential third element, the variable is updated. This strategy focuses on capturing the smallest possible candidate for the third element as the array is scanned. When an element larger than this updated variable is encountered, it indicates the presence of a valid "132" pattern, as it suggests a sequence where a smaller third element was found previously.

The steps are as follows:

1. **Initialization**:
    - A variable, `z`, is initialized with the smallest possible value. This variable is used to keep track of the potential 'z' value in our '132' pattern.
    - An ordered set (like a TreeSet in Java) is initialized to store and efficiently search for numbers.
2. **Iterating Backwards**:
    - We iterate through the array from the last element to the first. This reverse order is crucial for the logic of the algorithm.
3. **Updating 'z'**:
    - For each element (let's call it `current`), we check if it is smaller than `z`. If it is, it means we have found a valid '132' pattern, and the function returns `true`.
    - If `current` is not smaller than `z`, we search the ordered set for the largest number that is smaller than `current`. This number, if it exists, becomes the new `z`. This step ensures that `z` is always the best candidate for the middle element of our pattern.
4. **Maintaining the Ordered Set**:
    - The `current` element is then added to the ordered set. This set holds all the elements we have seen so far (in reverse order), allowing us to efficiently update `z` and check for the pattern.

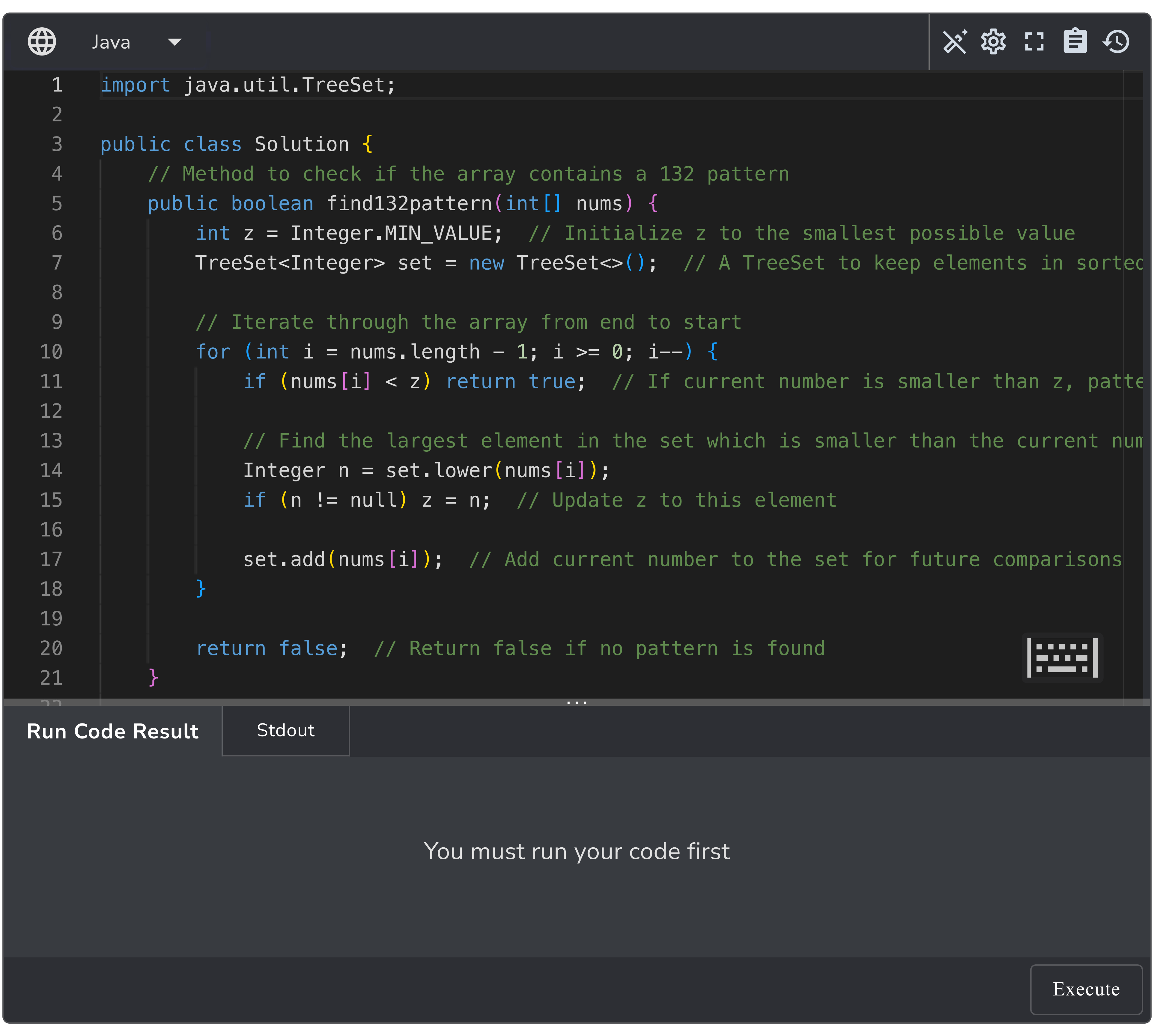
Example Walkthrough

Consider the example input: `[9, 11, 8, 9, 10, 7, 9]`. Let's walk through the algorithm with this input:

1. Initialize `z` to the smallest integer value. Initialize an empty ordered set.
2. Start iterating from the end of the array:
    - First element (from the end) is 9. It's not smaller than `z`, and there's no element in the set smaller than 9. Add 9 to the set.
    - Next element is 7. It's not smaller than `z`, and there's no element in the set smaller than 7. Add 7 to the set.
    - Next is 10. It's not smaller than `z`, but 9 is in the set and is smaller than 10, so update `z` to 9. Add 10 to the set.
    - Next is 9. It's not smaller than `z`, and there's no element in the set smaller than 9. Add 9 to the set.
    - Next is 8. It's smaller than `z` (which is 9), so we've found a '132' pattern. Return `true`.

In this example, when we reach the number 8, we already have 9 as our `z`, and since 8 is smaller than 9, it confirms the existence of the '132' pattern in the array.

Code



```java
import java.util.TreeSet;

public class Solution {
    // Method to check if the array contains a 132 pattern
    public boolean find132pattern(int[] nums) {
        int z = Integer.MIN_VALUE;  // Initialize z to the smallest possible value
        TreeSet<Integer> set = new TreeSet<>();  // A TreeSet to keep elements in sortec

        // Iterate through the array from end to start
        for (int i = nums.length - 1; i >= 0; i--) {
            if (nums[i] < z) return true;  // If current number is smaller than z, patte

            // Find the largest element in the set which is smaller than the current num
            Integer n = set.lower(nums[i]);
            if (n != null) z = n;  // Update z to this element

            set.add(nums[i]);  // Add current number to the set for future comparisons
        }

        return false;  // Return false if no pattern is found
    }
```

Complexity Analysis

- **Time Complexity**: O(n log n) as we iterate over all elements (O(n)) and each insertion or search in the ordered set/tree (or stack operations for some implementations) costs O(log n).
- **Space Complexity**: O(n), as we use an ordered set or stack to store elements.

← Previous
132 Pattern (Medium)
Next →
My Calendar I (Medium)
☐ Mark as Completed